

Variables, Data Types & Arithmetic Operators



- Produced Ms. Mairead Meagher,
 - by: Ms. Siobhán Roche.

Topics list

1. Variables.
2. Data Types.
3. Java's Primitive Data Types
 1. Whole numbers.
 2. Decimal numbers.
 3. Others.
4. Arithmetic operators.

Variables

In Programming, variables:

- are created (defined) in your programs.
- are used to store data (whose value can change over time).
- have a data type.
- have a name.
- are a VERY important programming concept.

Variable names...

- Are case-sensitive.
- Begin with either:
 - a **letter (preferable)**,
 - the dollar sign "\$", or
 - the underscore character "_".
- Can contain letters, digits, dollar signs, or underscore characters.
- Can be any length you choose.
- Must not be a **keyword or reserved word** e.g. int, while, etc.
- Cannot contain white spaces.

Variable names should be carefully chosen

- Use full words instead of cryptic abbreviations e.g.
 - variables named **speed** and **gear** are much more intuitive than abbreviated versions, such as **s** and **g**.
- If the name consists of:
 - only one word,
 - spell that word in all lowercase letters e.g. **ratio**.
 - more than one word,
 - capitalise the first letter of each subsequent word e.g. **gearRatio** and **currentGear**.
 - This is called **camelCase**

Kinds of variables so far

- Fields are one sort of variable.
 - They store values through the life of an object.
 - They are accessible throughout the class.
- Parameters are another sort of variable:
 - They receive values from outside the method.
 - They help a method complete its task.
 - Each call to the method receives a fresh set of values.
 - Parameter values are short lived.

Topics list

1. Variables.
2. Data Types.
3. Java's Primitive Data Types
 1. Whole numbers.
 2. Decimal numbers.
 3. Others.
4. Arithmetic operators.

Data Types

- In Java, when we define a variable, we **have** to give it a data type.
- The data type defines the **kinds of values** (data) that can be stored in the variable e.g.
 - - 456
 - 2
 - 45.7897
 - I Love Programming
 - S
 - true
- The data type also determines the **operations** that may be performed on it.

Data Types

- Java uses two kinds of data types:
 - **Primitive** types
 - **Object** types
- We are only looking at **Primitive** types now; we will cover Object types later in the module.

Topics list

1. Variables.
2. Data Types.
3. Java's Primitive Data Types
 1. Whole numbers.
 2. Decimal numbers.
 3. Others.
4. Arithmetic operators.

Java's Primitive Data Types

- Java programming language supports eight primitive data types.
- A primitive type is predefined by the language and is named by a reserved keyword.
- A primitive type is highlighted red when it is typed into the PDE e.g.

int numberOfItems;

boolean isAlive

float sizeOfSquare;

Topics list

1. Variables.

2. Data Types.

3. Java's Primitive Data Types

1. Whole numbers.

2. Decimal numbers.

3. Others.

4. Arithmetic operators.



Java's Primitive Data Types (whole numbers)

Type	Byte-size	Minimum value (inclusive)	Maximum value (inclusive)	Typical Use
byte	8-bit	-128	127	Useful in applications where memory savings apply.
short	16-bit	-32,768	32,767	
int	32-bit	-2,147,483,648	2,147,483,647	Default choice.
long	64-bit	-9,223,372,036,854,775,808	9,223,372,036,854,775,807	Used when you need a data type with a range of values larger than that provided by int.

TO HERE – add from 24 data types[®]

Scope highlighting

```
60 }
61 }
62 /**
63  * Print a ticket if enough money has been inserted, and
64  * reduce the current balance by the ticket price. Print
65  * an error message if more money is required.
66  */
67 public void printTicket()
68 {
69
70     if(balance >= price) {
71         // Simulate the printing of a ticket.
72         System.out.println("#####");
73         System.out.println("# The BlueJ Line");
74         System.out.println("# Ticket");
75         System.out.println("# " + price + " cents.");
76         System.out.println("#####");
77         System.out.println();
78
79         // Update the total collected with the price.
80         total = total + price;
81         // Reduce the balance by the price.
82         balance = balance - price;
83     }
84     else {
85         System.out.println("You must insert at least: " +
86             (price - balance) + " more cents.");
87     }
88 }
89 }
90 /**
91  * Return the money in the balance.
92  * The balance is cleared.
93  */
```

Scope and lifetime

- Each block defines a new scope.
 - Class, method and statement.
- Scopes may be nested:
 - statement block inside another block inside a method body inside a class body.
- Scope is static (textual).
- Lifetime is dynamic (runtime).

To think about

- How could we write a method to 'refund' an excess balance?
 - We need to return the value in balance.
 - We need to set the balance to zero.
- How can we do the first without losing the value in balance?

Unsuccessful attempt

```
public int refundBalance()  
{  
    // Return the amount left.  
    return balance;  
    // Clear the balance.  
    balance = 0;  
}
```

It looks logical, but the language does not allow it.

A method will terminate immediately after executing a return statement!

Local variables

- Methods can define their own, *local* variables:
 - Short lived, like parameters.
 - The method sets their values – unlike parameters, they do not receive external values.
 - Used for ‘temporary’ calculation and storage.
 - They exist only as long as the method is being executed.
 - They are only accessible from within the method.
 - They are defined within a particular *scope*.

Local variables

A local variable



No visibility
modifier

```
public int refundBalance()  
{  
    int amountToRefund;  
    amountToRefund = balance;  
    balance = 0;  
    return amountToRefund;  
}
```

Scope and lifetime

- The scope of a field is its whole class.
- The lifetime of a field is the lifetime of its containing object.
- The scope of a local variable is the block in which it is declared.
- The lifetime of a local variable is the time of execution of the block in which it is declared.

Questions?

